

# Оформление модуля(драйвера) в системе

## Заголовочный файл

```
#pragma once

/* если модуль реализует какие либо системные интерфейсы, включить их
заголовки (см. /HDL/Ifaces) например #include "hdl_nvm.h" */

/* определить размерность дескриптора */
#define HDL_BUTTON_VAR_SIZE    40

/* определить статическую конфигурацию */
typedef struct {
    uint32_t debounce_delay;
    uint32_t hold_delay;
} hdl_button_config_t;

/* Определить интерфейс модуля или использовать стандартный (см. /HDL/Ifaces)
*/
typedef hdl_btn_state_t (* hdl_button_state_get_t)(const void *);
typedef uint8_t (* hdl_button_sw_action_t)(const void *);

/* Первый метод в интерфейсе всегда hdl_module_initializer_t, остальные
произвольно */
typedef struct {
    hdl_module_initializer_t init;
    hdl_event_subscribe_t subscribe;
    hdl_button_state_get_t state_get;
    hdl_button_sw_action_t press;
    hdl_button_sw_action_t click;
    hdl_button_sw_action_t release;
} hdl_button_iface_t;

/* Создать тип модуля описать необходимые зависимости.
используй специальный макрос hdl_module_new_t для определения модуля.
Макрос принимает в качестве параметров:
- имя нового типа
- размер резервируемой памяти ОЗУ
- тип статической конфигурации
- тип реализуемого интерфейса.
*/
/* depends on:
    hdl_sdio_t
    hdl_time_counter (ms)
    hdl_gpio_t      (card detect)
*/
/* depends on:
    gpio
    time_counter
```

```

*/
hdl_module_new_t(hdl_button_t, HDL_BUTTON_VAR_SIZE, hdl_button_config_t*,
hdl_button_iface_t);

/* Определить экспортируемый интерфейс модуля */
extern const hdl_button_iface_t hdl_button_iface;

/* Для удобства определить вызовы для интерфейса модуля */
__STATIC_INLINE hdl_btn_state_t hdl_button_state_get(const void *desc) {
    MODULE_ASSERT(desc, HDL_BTN_UNKNOWN);
    return ((hdl_button_t *)desc)->iface->state_get(desc);
}

__STATIC_INLINE uint8_t hdl_button_sw_press(const void *desc) {
    MODULE_ASSERT(desc, HDL_FALSE);
    return ((hdl_button_t *)desc)->iface->press(desc);
}

__STATIC_INLINE uint8_t hdl_button_sw_click(const void *desc) {
    MODULE_ASSERT(desc, HDL_FALSE);
    return ((hdl_button_t *)desc)->iface->click(desc);
}

__STATIC_INLINE uint8_t hdl_button_sw_release(const void *desc) {
    MODULE_ASSERT(desc, HDL_FALSE);
    return ((hdl_button_t *)desc)->iface->release(desc);
}

__STATIC_INLINE void hdl_button_subscribe(const void *desc, hdl_delegate_t
*delegate) {
    MODULE_ASSERT(desc, );
    ((hdl_button_t *)desc)->iface->subscribe(desc, delegate);
}

```